

KMP算法讲义

1.最简单的是暴力算法

暴力算法就是将两个字符串依次匹配。

```
1  #include<stdio.h>
2  #include<string.h>
3
4  /*暴力*/
5  int main(){
6      char source[] ="This is data structure,please find data"; //主
      字符串
7      char *find = "data";          // 模式字符串
8
9      int where = 1;
10     int lenSource = strlen(source);
11     int lenFind = strlen(find);
12     int i = 0 ,j = 0;
13     for(;i <= lenSource - lenFind; i++){
14         for(j = 0;j < lenFind; j++){
15             if(source[i+j] != find[j]){
16                 break;
17             }
18         }
19         if(j == lenFind){
20             printf("第%d个匹配的位置: %d\n",where,i);
21             where++;
22         }
23     }
24
25     return 0;
26 }
```

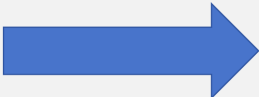
这种暴力算法，相当于将**模式串**不断地平移。完全匹配就输出。

This is **data** structure,please find **data**

data

data

data



2.稍微优化一下

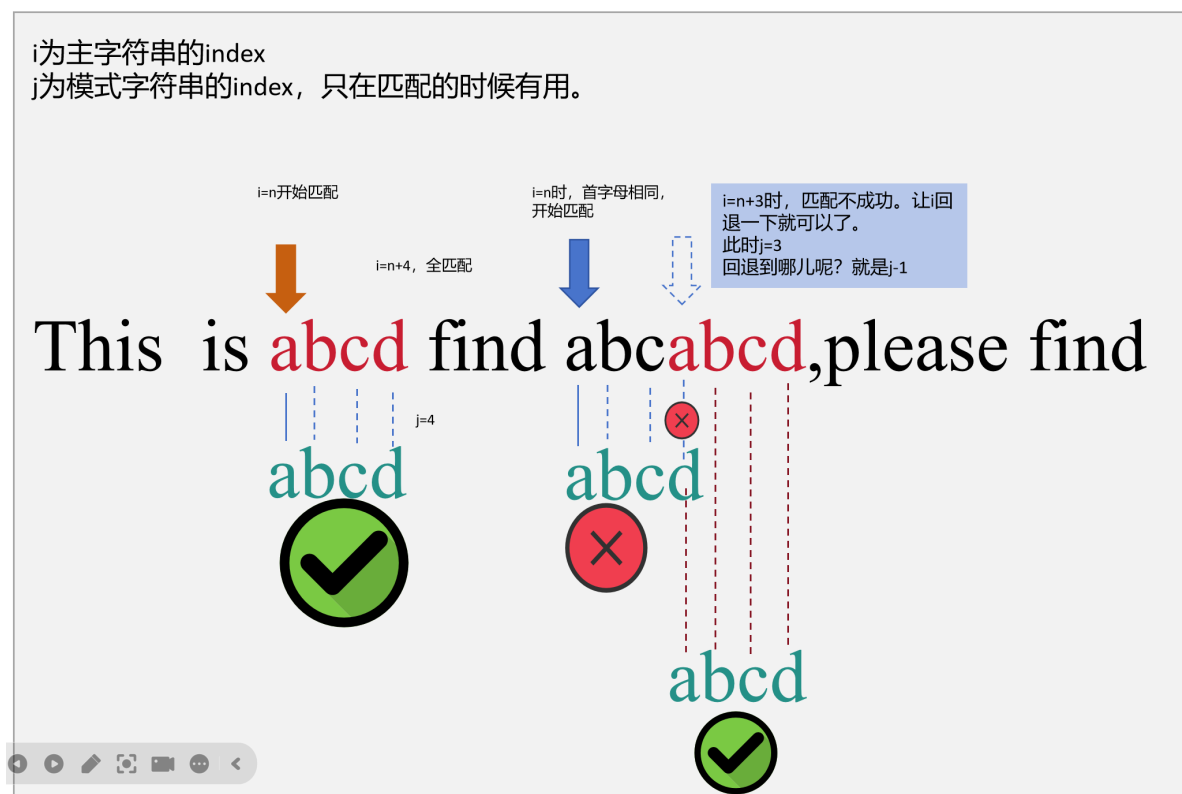
将暴力算法中的“依次”，改成首字母相同后，再来匹配。因为若首字母都不匹配，不可能匹配成功。

```
1  #include<stdio.h>
2  #include<string.h>
3
4  /*简单优化*/
5  int main(){
6      char source[] ="This is data dastructure,please find data";
7      char *find = "data";
8      /**
9       * 1.找出首字母相同的字母
10      * 2.然后往下匹配
11      */
12     int where = 1;
13     int lenSource = strlen(source);
14     int lenFind = strlen(find);
15     int i = 0, j = 0;
16
17     for(;i <= lenSource-lenFind; i++){
18         if(source[i] != find[0]){
19             continue; //找到首字母相同，再开始匹配
20         }
21         for(j = 0;j<lenFind;j++){
22             if(source[i+j] != find[j]){
23                 break;
24             }
25         }
26         if(j == lenFind ){
27             printf("第%d个匹配的位置: %d\n",where,i);
28             where++;
29         }
30     }
31 }
```

可用一下图片来解释上面的代码。



上面的代码中，用了循环嵌套，时间复杂度为 $M \times N$ 。其实完全可以用一次循环来搞定。我们将主字符串和模式字符串修改一下，来举个例子。



实现代码如下：

```
1  #include<stdio.h>
2  #include<string.h>
3
4  /*简单优化*/
5  int main(){
6      char source[] ="This is abcd find abcab cd, please find ";
7      char *find = "abcd";
8      /**
9       * 1.找出首字母相同的字母
10      * 2.然后往下匹配
```

```

11      * 3.匹配成功就划走
12      * 4.匹配不成功，就回退.回退到不匹配的index-1
13      */
14      int where = 1;
15      int lenSource = strlen(source);
16      int lenFind = strlen(find);
17      int i = 0, j = 0;
18
19      for(;i < lenSource-lenFind; i++){
20
21          /** 由于只用1个循环，下面的代码是不能用的*/
22          /*
23          if(source[i] != find[0]){
24              continue; //找到首字母相同，再开始匹配
25          }*/
26
27          /**现在字符不相同分为两种情况：
28          * 1.还没有开始匹配，此时j始终保持0
29          * 2.已经开始匹配了，但中间不相同，j此时不是0了。
30          */
31          if(source[i] != find[j]){
32              i = (j > 0) ? i - j + 1 : i;
33              j = 0;
34          }else{
35              // 这是两个字符串中的字符相同，首字母相同也包含在内。
36              j++;
37          }
38
39          if(j == lenFind ){
40              printf("第%d个匹配的位置: %d\n",where,i-lenFind);
41              where++;
42          }
43      }
44  }

```

3.KMP算法

这个算法由[高德纳](#)和[沃恩·普拉特](#)在1974年构思，同年[詹姆斯·H·莫里斯](#)也独立地设计出该算法，最终三人于1977年联合发表。

它由两个要点：

- i不回退。因为一旦回退就意味时间复杂度的提高，万一模式字符串非常长——这种情况在大数据场景下是很常见的，就浪费了。
- 时间复杂度为M+N，不能比这个还高。实际上，这个复杂度就意味着i是不能回退的。

运动是相对的，既然i不回退，那我们可以将模式字符串向右平移来匹配。平移多少呢？从下面的案例1可以看出，模式串整体平移就行了。

案例1



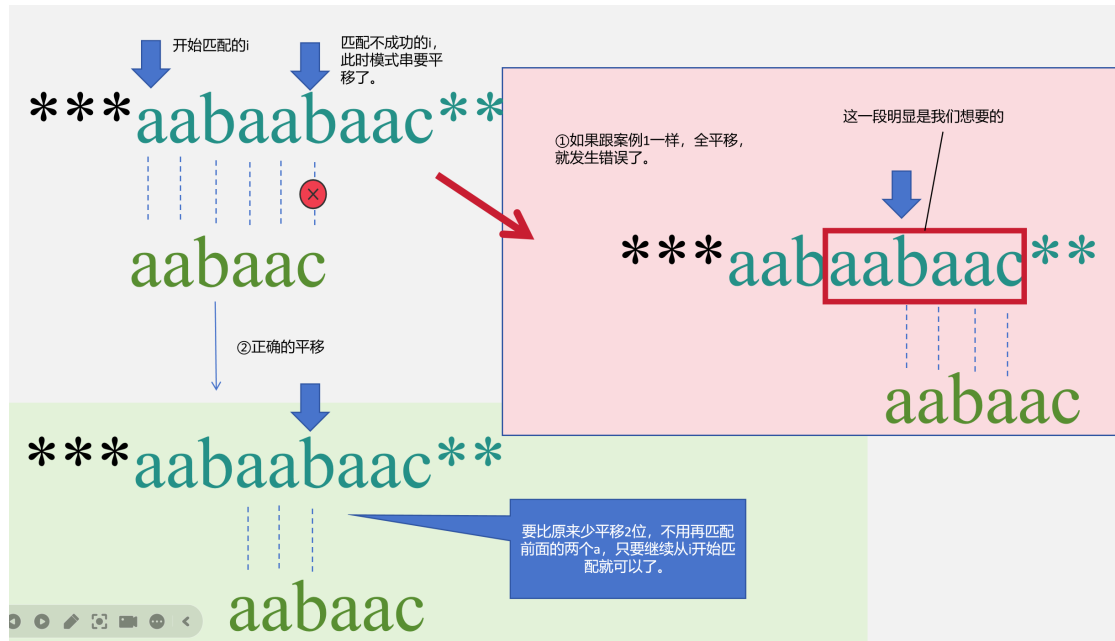
代码如下：

```
1  #include<stdio.h>
2  #include<string.h>
3
4  /*简单优化*/
5  int main(){
6      char source[] ="This is abcd find abcabcd,please find ";
7      char *find = "abcd";
8      int where = 1;
9      int lenSource = strlen(source);
10     int lenFind = strlen(find);
11     int i = 0, j = 0;
12     for(;i < lenSource-lenFind; i++){
13         if(source[i] != find[j]){
14             i = (j > 0) ? i - 1 : i;
15             // 这里的i不能回退，但需要等待一下，因为我们用了for语句，里面的i++总是要加一次的
16             // 现在i = i - 1 ,则意味上面的i++ 没加没减
17             j = 0;    // 这个j=0，因为模式串平移了。
18         }else{
19             j++;
20         }
21
22         if(j == lenFind ){
23             printf("第%d个匹配的位置: %d\n",where,i-lenFind);
24             where++;
25         }
26     }
27 }
```

但是，案例1中的代码不回退，也得到了正确的结果，是因为整个模式字符串中，没有一个字符与模式字符串的首字母相同。

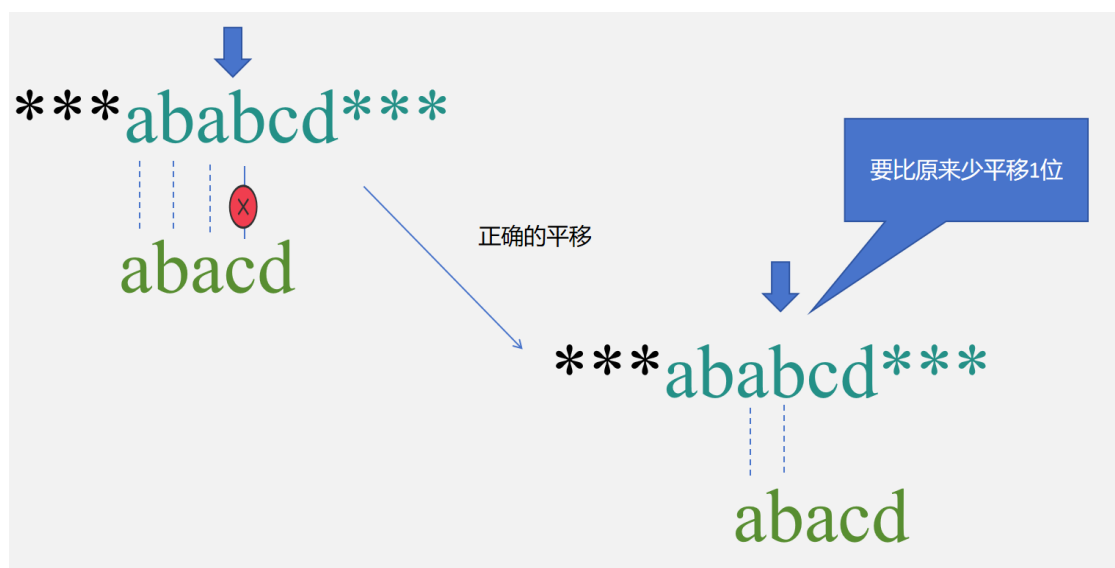
如果，有一个字符与模式首字符串相同，就会出问题。如下图。

案例2

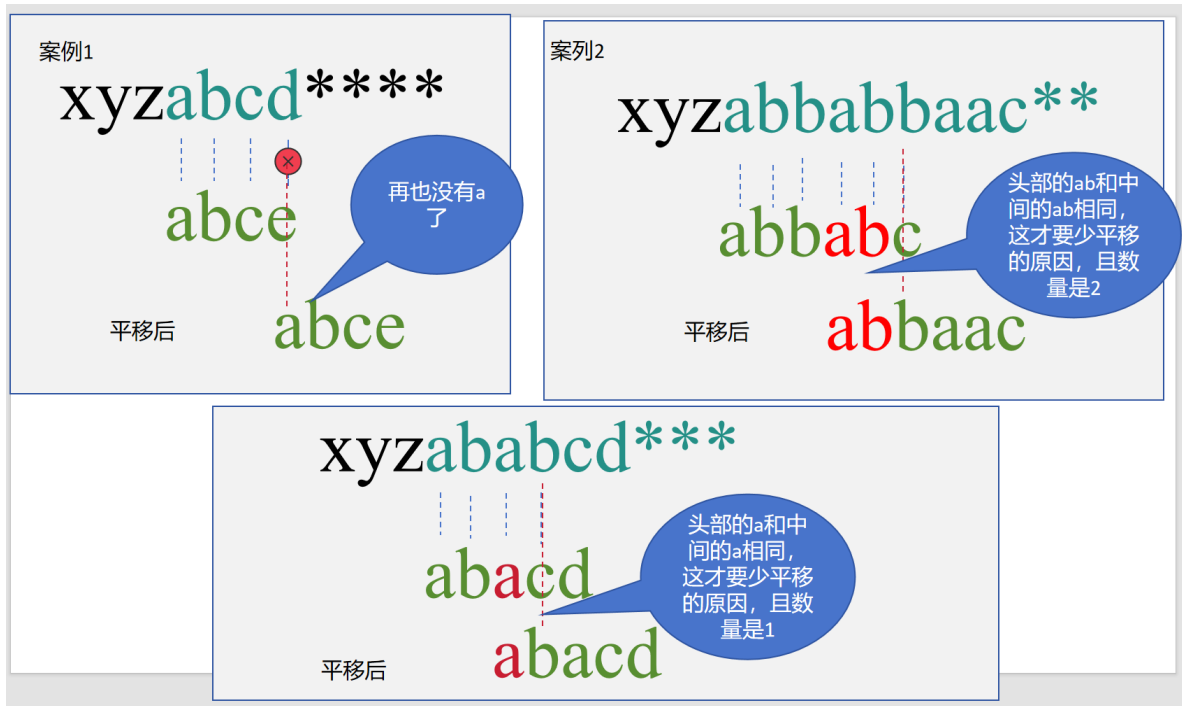


案例2中，出现不匹配的位置是在模式串的末尾，也有可能出现在模式串的中间。请看案例3

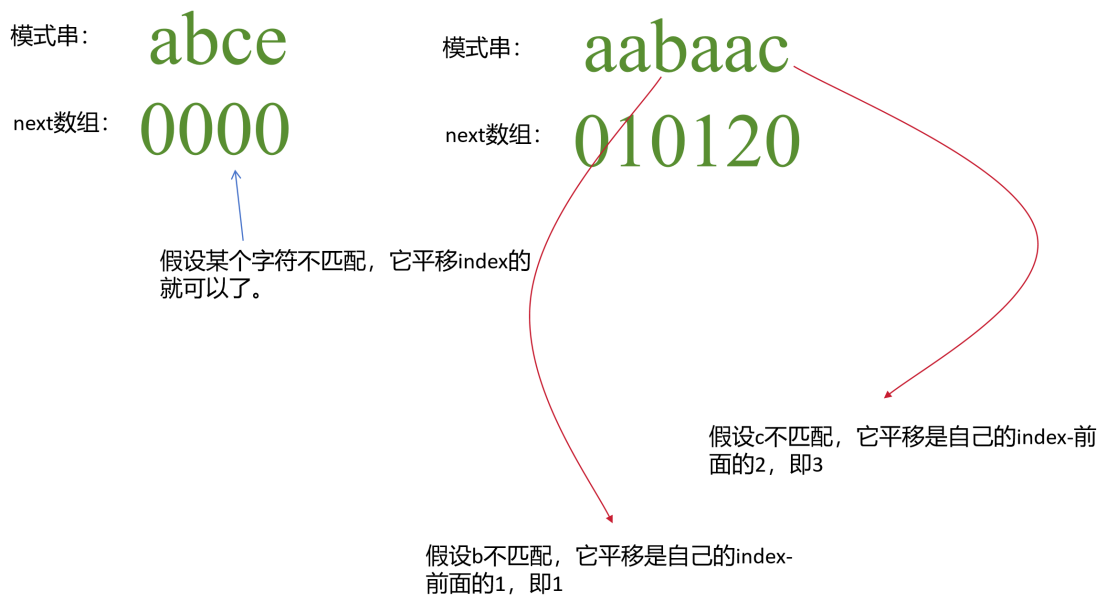
案例3



现在，我们就需要从以上案例中找出少平移的规律



由此可见，少平移是模式串带来的，跟主串没关系。少平移的数量取决于模式串的中间，最长的与头部相同的字符串长度。为了记录这一信息，KMP算法准备了next数组。



实现如下：

```

1 void calNext(int *next,char *find, int length){
2
3     if(next == NULL || find == NULL)
4         return;
5
6     int i = 1;
7     for(;i<length;i++){
8         if(find[i] == find[next[i-1]]){
9             next[i] = next[i-1]+1;
10        }
11    }
12 }

```

最终实现代码如下。我用for实现的，网上有更简洁的代码，用while实现的。但道理都差不多。

```

1 int main(){
2     // char source[] ="This is abcabcd dastructure,please find
    abcabcd";
3     // char *find = "abcabcd";
4
5     char source[] ="xyzaabaabaacd";
6     char *find = "aabaac";
7     int where[100] = {0};
8     int whereIndex = 0;
9     int lenSource = strlen(source);
10    int lenFind = strlen(find);
11    int *next = (int*) malloc(lenFind * sizeof(int));
12    memset(next,0, lenFind * sizeof(int));
13    calNext(next,find,lenFind);    // 获得next数组
14
15
16    int i = 0, j = 0 ;
17
18    for(;i < lenSource;i++){
19        if(source[i] != find[j]){
20            i = (j > 0) ? i - 1 : i;    // 此处i是等待，不是回退，若用
    while可消除此代码
21            j = (j > 0) ? next[ j-1 ] : 0;
22            // 案例1时，j直接等于0，本是上它next数组的的值为0。
23            // 案例2,3中，j不能再直接被赋值为0，是next数组决定的。
24        }else{
25            j++;
26        }
27
28        /**这里做了一点改变，即将匹配到的位置保存了起来。*/
29        if(j == lenFind){

```



```
30         where[whereIndex] = i - lenFind + 1;
31         whereIndex++;
32         j = 0;
33     }
34 }
35 for(i = 0; i < whereIndex; i++){
36     printf("第%d个匹配的位置:%d \n", i, where[i]);
37 }
38
39 free(next);
40 return 0;
41 }
```

总结与心得

记得给423上Java课时，也讲了KMP算法，当时用Java来实现的，讲成什么样，不记得了，但记得用Java写的代码比上面的要简练——道理是一样的。但我不愿意再翻出来了，宁愿再写一次。对于初学者，千万不要以为，代码你看懂了就行，而是认真写出来，踏踏实实地锻炼自己的编程思维，后面再学会使用AI，肯定有一碗饭吃的。